

Project Report

On

DATABASE IMPLEMENTATION IN

JDBC

(CSE-3488)

Designing a Scalable Online Bookstore Management System using JDBC with Full Transaction Integrity and Query Optimization (Apache Derby)



Submitted by:

<u>Name</u>	<u>Registration Number</u>
Subham Kumar Sahoo	2341019060

B. Tech. CSE(AI-ML)
6th Semester
Section - 23412C3

Index

SL. No.	Chapter No.	Title of the Chapter	Page No.
1	1	Abstract	3
2	2	Objective	4
3	3	Problem Statement	5
4	4	Project Details	6–9
5	5	Hardware & Software Requirements	10
6	6	Project Implementation	11–50
7	7	Snapshots	51–54
8	8	Conclusion	55
9	9	References	56

Chapter 1: Abstract

This project details the design and development of a console-based Online Bookstore Management System, constructed using Java JDBC with Apache Derby operating in embedded mode. The system showcases a comprehensive, end-to-end database application structured across five well-defined architectural tiers: connection lifecycle management, ACID-compliant transaction control, business logic execution, performance benchmarking, and an interactive command-line user interface.

The central emphasis of this project lies in rigorous transaction management. Multi-step purchase workflows involving the Customers, Products, and Orders tables are encapsulated within atomic transaction boundaries, leveraging explicit commit and rollback mechanisms. Named savepoints facilitate selective partial recovery within a transaction without necessitating a complete rollback. Data correctness is enforced through primary keys, foreign key relationships, unique constraints on critical fields, and check constraints on column values, ensuring that the system maintains integrity even when failures occur mid-operation.

A dedicated performance evaluation module empirically measures four key JDBC access strategies: single-row versus batch inserts, full-table scans versus B-tree indexed queries, raw Statement concatenation versus compiled PreparedStatement execution, and per-statement commits versus grouped batched commits. The benchmarking results demonstrate that PreparedStatement delivers approximately 30x greater throughput compared to string-built Statement queries, batch inserts surpass row-by-row inserts by a factor of nearly 1.75x, and batched commits reduce overhead by a factor of roughly 24x relative to per-operation commits.

The project confirms that correctness and efficiency are not mutually exclusive objectives. PreparedStatement provides both SQL injection prevention and superior execution speed, while structured transaction scoping simultaneously ensures data integrity and optimized I/O performance — qualities essential for a production-ready bookstore management environment.

Chapter 2: Objective

The primary objective of this project is to design and implement a complete, layered JDBC application that demonstrates professional-grade database programming practices through a real-world Online Bookstore Management System. The specific objectives are as follows:

1. Database Design and Initialization

To design a normalized relational schema consisting of Customers, Products, and Orders tables, each equipped with appropriate primary keys, foreign key constraints, unique indexes, and check constraints. The schema is to be initialized programmatically using an embedded Apache Derby database, with seed data inserted automatically on first application launch.

2. Transaction Management and ACID Compliance

To enforce explicit transaction control across all data-modifying operations by disabling auto-commit, wrapping multi-step business workflows in atomic commit and rollback boundaries, and using named savepoints to enable partial recovery without discarding completed intermediate steps. The system must concretely demonstrate all four ACID properties — Atomicity, Consistency, Isolation, and Durability — via operational code and deliberate failure scenarios.

3. Robust Error Handling and Resource Management

To ensure that all JDBC resources — connections, prepared statements, and result sets — are properly acquired and deterministically released using try-with-resources constructs, and that all constraint violations and SQL exceptions are gracefully intercepted, logged, and recovered without leaving the database in a partial or corrupted state.

4. Core Application Functionality

To develop a menu-driven command-line interface enabling users to register customers, manage the product catalog, process purchase and refund transactions, query active and pending orders, and execute administrative operations — all backed exclusively by parameterized PreparedStatement queries throughout.

5. Performance Benchmarking and Empirical Analysis

To engineer and run a systematic benchmarking framework that measures and contrasts the execution time and throughput of four JDBC access patterns — individual versus batch inserts, full-table scans versus indexed lookups, Statement versus PreparedStatement, and per-operation versus batched commits — across multiple runs with warm-up phases to produce statistically reliable and reproducible results.

6. Documentation and Insight

To produce a structured analytical document explaining, with reference to Derby's internal engine architecture, why certain JDBC patterns outperform others, how transaction boundaries safeguard data integrity during failure scenarios, and what trade-offs exist between safety guarantees and raw throughput in a production JDBC application.

Chapter 3: Problem Statement

Modern e-commerce bookstore systems are required to reliably process customer purchases, maintain accurate inventory counts, and record complete order histories — all while ensuring that no product is simultaneously listed as available and sold out, no customer exceeds their account limits, and no partial operation leaves the database in an inconsistent state. These requirements become non-trivial when multiple interdependent database tables must be updated atomically as a single coherent business transaction.

Raw JDBC, while highly capable, provides no automatic safety mechanism. In its default auto-commit mode, every individual statement is immediately and permanently persisted, which means a failure occurring between two related updates can corrupt the database irreversibly. Developers must therefore take full responsibility for transaction boundaries, rollback logic, savepoint management, and constraint handling. Without this discipline, even a straightforward purchase operation touching three tables becomes a latent source of silent data corruption.

Beyond correctness, real-world database applications must address performance. Naive JDBC usage — string-concatenated statements, per-row commits, and unindexed full-table scans — can degrade system throughput by orders of magnitude compared to optimized patterns. Yet the interaction between safety mechanisms such as transactions and prepared statements and their performance implications is rarely demonstrated empirically in a controlled, reproducible setting.

This project addresses both challenges in tandem. It develops a fully functional Online Bookstore Management System using Java JDBC and Apache Derby that enforces ACID transaction semantics across all data-modifying operations, while simultaneously benchmarking the performance impact of four key JDBC design decisions: insert strategy, query strategy, statement type, and transaction granularity. The goal is to demonstrate through working code and measured results that correctness and efficiency are complementary rather than competing concerns in a well-designed JDBC application.

Chapter 4: Project Details

4.1 Technology Stack

Component	Technology
Programming Language	Java 11+
Database Engine	Apache Derby 10.14.2+ (Embedded Mode)
Database Access	Java JDBC (java.sql.*)
Interface	Console-driven CLI
Build Tool	javac (manual compilation)
Operating System	Ubuntu/Debian Linux

4.2 System Architecture

The application is organized into five distinct layers following a strict separation-of-concerns design principle:

```
OnlineBookstoreSystem/
├── src/main/java/com/bookstore/
│   ├── model/                # Data model classes
│   │   ├── Customer.java
│   │   ├── Product.java
│   │   └── Order.java
│   ├── connection/          # Derby DB lifecycle
│   │   └── ConnectionManager.java
│   ├── transaction/         # ACID transaction boundaries
│   │   └── TransactionService.java
│   ├── business/            # CRUD / domain logic
│   │   └── BusinessLogic.java
│   ├── benchmark/           # Performance evaluator
│   │   └── PerformanceEvaluator.java
│   └── ui/                   # CLI interface
│       └── MainApp.java
├── out/                      # Compiled .class files
├── docs/
│   └── analysis.md           # Transaction & performance analysis
├── performance_report.csv    # Generated after benchmarks
└── BookstoreDB/              # Derby database directory (auto-created)
```

4.3 Database Schema

The system uses three normalized relational tables designed to enforce referential integrity and prevent invalid data states at the storage engine level.

Customers Table

Stores registered bookstore customers. The ActiveOrders counter is maintained transactionally and is constrained to never go negative.

```
CREATE TABLE Customers (
  CustomerID  INTEGER NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  FullName    VARCHAR(100) NOT NULL,
```

```

Email      VARCHAR(150) NOT NULL UNIQUE,
Phone      VARCHAR(20),
RegisterDate DATE NOT NULL,
ActiveOrders INTEGER DEFAULT 0 CHECK (ActiveOrders >= 0)
)

```

Products Table

Stores the book product catalog with stock tracking. StockCount is decremented on purchase and incremented on refund, while the InStock flag reflects overall availability.

```

CREATE TABLE Products (
  ProductID    INTEGER NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  Title        VARCHAR(200) NOT NULL,
  Author       VARCHAR(100) NOT NULL,
  ISBN         VARCHAR(20) NOT NULL UNIQUE,
  Genre        VARCHAR(50),
  Price        DECIMAL(8,2) DEFAULT 0.00 CHECK (Price >= 0),
  InStock      SMALLINT DEFAULT 1 CHECK (InStock IN (0,1)),
  TotalStock   INTEGER DEFAULT 1 CHECK (TotalStock > 0),
  StockCount   INTEGER DEFAULT 1 CHECK (StockCount >= 0)
)

```

Orders Table

Records every purchase event with a status lifecycle of PENDING → COMPLETED or CANCELLED. Foreign keys enforce that every order references a valid customer and product.

```

CREATE TABLE Orders (
  OrderID      INTEGER NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  CustomerID   INTEGER NOT NULL REFERENCES Customers(CustomerID),
  ProductID    INTEGER NOT NULL REFERENCES Products(ProductID),
  OrderDate    DATE NOT NULL,
  DeliveryDate DATE NOT NULL,
  CompletedDate DATE,
  Status       VARCHAR(10) DEFAULT 'PENDING'
              CHECK (Status IN ('PENDING', 'COMPLETED', 'CANCELLED'))
)

```

4.4 Indexes

Index Name	Table	Column	Purpose
idx_products_isbn	Products	ISBN	Fast ISBN-based product lookup
idx_orders_customer	Orders	CustomerID	Fast order lookup per customer
idx_orders_completed	Orders	CompletedDate	Fast completion date filtering
idx_orders_status	Orders	Status	Fast pending/completed order filtering
idx_orders_delivery	Orders	DeliveryDate	Fast delivery date range queries

4.5 Core Components

1. ConnectionManager

Implemented as a Singleton that manages the full lifecycle of the embedded Derby database. Responsible for: opening and reusing a single physical connection, detecting schema existence via DatabaseMetaData, creating tables and indexes on first run, inserting seed data within a transaction, verifying indexes programmatically, and performing a clean Derby shutdown on exit.

2. TransactionService

Handles all operations requiring explicit multi-statement transaction boundaries. Its two primary operations are:

- `processPurchase(productId, customerId)`: A four-step atomic transaction that verifies product availability, inserts an Orders record, sets a named savepoint, updates `Products.StockCount`, and updates `Customers.ActiveOrders`. All steps commit together or roll back entirely on any failure.
- `processRefund(orderId)`: Retrieves the order record, updates its status to CANCELLED, restores `Products.StockCount`, decrements `Customers.ActiveOrders`, and commits — all within a single transaction boundary.

3. BusinessLogic

Provides all CRUD operations using PreparedStatements with try-with-resources for proper resource cleanup. Covers customer registration and retrieval, product catalog management including ISBN-based lookup, order querying by customer and pending status, bulk status updates for delayed orders, and a duplicate order guard that prevents a customer from purchasing the same product twice simultaneously.

4. PerformanceEvaluator

A self-contained benchmarking module that creates its own isolated benchmark tables. Implements four comparative test suites: Insert Strategy, Query Strategy, Statement Type, and Transaction Granularity. Each suite includes a warm-up phase to stabilize JVM JIT compilation and Derby's buffer cache before timing begins.

5. MainApp

The application entry point and CLI orchestrator. Initializes all services, presents a numbered main menu, and routes user input to the appropriate service layer — all within a single try-catch-finally block guaranteeing Derby shutdown even on fatal errors.

4.6 Transaction Management Detail

The `processPurchase()` operation is the most critical transaction in the system. Its execution flow is as follows:

```
BEGIN TRANSACTION (auto-commit = false)
|
|— Step 1: Verify Products.StockCount > 0
|— Step 2: INSERT INTO Orders → generate OrderID
|— SAVEPOINT 'AFTER_ORDER_INSERT'
|— Step 3: UPDATE Products SET StockCount = StockCount - 1
|— Step 4: UPDATE Customers SET ActiveOrders = ActiveOrders + 1
|
|— SUCCESS → COMMIT (all four steps persist)
|— FAILURE → ROLLBACK TO SAVEPOINT or full ROLLBACK
```


This pattern guarantees that the database can never reach a state where an order record exists but the product's stock count has not been decremented, or vice versa.

4.7 Seed Data

The database is pre-populated on first run with 5 customers and 10 products spanning Technology, Fiction, History, Science, and Self-Help genres, providing a realistic working dataset for all CLI operations and benchmarks without requiring manual data entry.

4.8 Performance Summary

Benchmark	Result
Batch vs Individual Insert (10,000 rows)	Batch is ~1.75x faster
Indexed vs Full-Table Scan	Index is ~2.5x faster
PreparedStatement vs Statement (500 ops)	PreparedStatement is ~30x faster
Batched Commit vs Per-op Commit (100 ops)	Batched commit is ~24x faster

4.9 Build and Run

Install Dependencies

```
sudo apt install default-jdk libderby-java
```

Compile

```
find src -name "*.java" > sources.txt
mkdir -p out
javac -cp /usr/share/java/derby.jar -d out @sources.txt
```

Run

```
java -cp "out:/usr/share/java/derby.jar" com.bookstore.ui.MainApp
```

The Derby database directory (BookstoreDB/) is created automatically on first run. No manual schema setup is required.

Chapter 5: Hardware & Software Requirements

5.1 Hardware Requirements

Component	Minimum	Recommended	Optimal
Processor	Dual-core 1.8 GHz x86-64	Quad-core 2.5 GHz x86-64	Hexa-core 3.0+ GHz (Intel i5/i7, AMD Ryzen 5/7)
RAM	2 GB	4 GB	8 GB or more
Storage	500 MB free (HDD)	1 GB free (SSD preferred)	2 GB free on SSD (for fast Derby I/O)
Display	800 × 600	1280 × 720	1920 × 1080 (for IDE comfort)
Network	Not required (embedded mode)	Not required (embedded mode)	LAN/localhost (for network mode testing)

5.2 Software Requirements

Component	Minimum	Recommended	Optimal
Operating System	Windows 7 / Ubuntu 16.04 / macOS 10.12	Windows 10 / Ubuntu 20.04 / macOS 12	Windows 11 / Ubuntu 22.04 LTS / macOS 14
Java (JDK)	JDK 8 (Java SE 8)	JDK 11 (LTS)	JDK 17 or JDK 21 (LTS)
Apache Derby	Derby 10.14 (JDK 8 compatible)	Derby 10.15 (JDK 11 compatible)	Derby 10.16+ (JDK 17/21 compatible)
IDE / Editor	Any text editor + CLI	Eclipse IDE or VS Code with Java Extension	IntelliJ IDEA (Community or Ultimate)
Build Tool	javac + manual classpath	Apache Maven 3.6+	Apache Maven 3.9+ or Gradle 8+
Version Control	None (local copy)	Git 2.x	Git 2.x + GitHub / GitLab remote
Derby JDBC Driver	derby.jar on classpath	derby.jar via Maven dependency	derby.jar + derbytools.jar via Maven

Chapter 6: Project Implementation

Phase 1 — Database Initialization & Schema Design

Overview

Phase 1 establishes the database foundation of the Online Bookstore Management System. Everything in this phase is handled automatically by the ConnectionManager on first application startup, requiring no manual setup from the user.

Database Engine

The system uses Apache Derby in embedded mode, meaning the database runs entirely within the same JVM process as the application. There is no separate server to install or configure. The database is stored as a local directory and is created automatically on first run.

Connection Strategy

A single shared database connection is maintained throughout the application's lifetime using the Singleton design pattern. All layers of the application share this one connection, avoiding the overhead of repeatedly opening and closing connections.

Schema Existence Check

Every time the application starts, it first checks whether the schema already exists by querying Derby's internal system catalog. If the tables are found, initialization is skipped entirely. If not, the full setup sequence runs, ensuring correct behavior on both first and subsequent launches.

Schema Design

Three normalized tables are created:

- Customers — stores registered users with a unique email constraint and an active order counter constrained to never go negative.
- Products — stores the catalog with ISBN uniqueness, total stock count, and available stock tracking.
- Orders — records every purchase event, linking customers and products via foreign keys, with a status lifecycle of PENDING, COMPLETED, or CANCELLED enforced by a check constraint.

Indexes

Index	Table	Column
idx_products_isbn	Products	ISBN
idx_orders_customer	Orders	CustomerID
idx_orders_completed	Orders	CompletedDate
idx_orders_status	Orders	Status
idx_orders_delivery	Orders	DeliveryDate

Seed Data

Five customers and ten products across five genres — Technology, Fiction, History, Science, and Self-Help — are inserted on first run. The insertion is wrapped in a transaction that either commits all records together or rolls back entirely.

Graceful Shutdown

On exit, the ConnectionManager closes the JDBC connection and issues a Derby shutdown command that flushes all pending writes and releases all file locks. Derby confirms successful shutdown by returning SQLState 08006, which the application recognizes as expected behavior.

ConnectionManager.java — Code

```
package com.bookstore.connection;

import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConnectionManager {

    private static final String DB_URL =
        "jdbc:derby:BookstoreDB;create=true";
    private static final String SHUTDOWN_URL =
        "jdbc:derby:BookstoreDB;shutdown=true";
    private static final String DRIVER_CLASS =
        "org.apache.derby.jdbc.EmbeddedDriver";

    private static ConnectionManager instance;
    private Connection connection;

    private ConnectionManager() {}

    /** Singleton accessor */
    public static ConnectionManager getInstance() {
        if (instance == null) instance = new ConnectionManager();
        return instance;
    }

    public Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed()) {
            try { Class.forName(DRIVER_CLASS); }
            catch (ClassNotFoundException e) {
                throw new SQLException("Derby driver not found.", e);
            }
            connection = DriverManager.getConnection(DB_URL);
        }
        return connection;
    }

    public void initializeDatabase() throws SQLException {
        Connection conn = getConnection();
        if (!tablesExist(conn)) {
            createTables(conn);
            createIndexes(conn);
        }
    }
}
```

```

        insertSeedData(conn);
    }
}

private boolean tablesExist(Connection conn) throws SQLException {
    DatabaseMetaData meta = conn.getMetaData();
    try (ResultSet rs = meta.getTables(null, "APP", "CUSTOMERS", new
String[]{"TABLE"})) {
        return rs.next();
    }
}

public void shutdown() {
    try { if (connection != null && !connection.isClosed())
connection.close(); }
    catch (SQLException ignored) {}
    try { DriverManager.getConnection(SHUTDOWN_URL); }
    catch (SQLException e) {
        if ("08006".equals(e.getSQLState()))
            System.out.println("[ConnectionManager] Derby shutdown
cleanly.");
    }
}
}

```

Phase 2 — Transaction Management Implementation

Overview

Phase 2 introduces explicit transaction control ensuring all data-modifying operations that span multiple tables either complete fully or leave the database completely unchanged. It demonstrates all four ACID properties through real operations and deliberate failure scenarios.

Why Transaction Management is Necessary

A bookstore purchase is not a single database action. When a customer places an order, three separate tables must be updated: a new record must be created in the Orders table, the available stock count in the Products table must be decremented, and the active order counter in the Customers table must be incremented. Without explicit transaction control, Derby's default auto-commit behaviour permanently saves each individual update the moment it executes, making partial failures impossible to undo.

Core Transaction — Processing a Purchase

The purchase processing operation consists of four sequential steps:

1. **Availability Check:** Verifies the requested product has at least one unit in stock before any data is modified.
2. **Order Record Creation:** Inserts a new record into the Orders table capturing the customer ID, product ID, order date set to today, and delivery date set to seven days from today.
3. **Product Stock Update:** Decrements the StockCount in the Products table by one. If this brings the count to zero, the InStock flag is also set to zero.
4. **Customer Order Counter Update:** Increments the ActiveOrders counter in the Customers table by one.

If all four steps complete without error, a single commit is issued. If any step throws an exception, a full rollback undoes every change, returning all three tables to exactly the state they were in before the operation began.

Savepoint — Partial Rollback Support

Between Step 2 and Step 3, after the order record has been successfully inserted, a named savepoint `AFTER_ORDER_INSERT` is set. This acts as a bookmark within the transaction, enabling selective rollback to this point without discarding all earlier work. If Step 4 fails, the transaction can roll back to the savepoint and attempt corrective action before deciding whether to commit or fully roll back.

ACID Properties Demonstrated

Property	How Phase 2 Demonstrates It
Atomicity	<code>processPurchase()</code> and <code>processRefund()</code> commit all steps together or roll back all steps together — there is no partial state
Consistency	Check constraints, foreign keys, and unique constraints prevent the database from ever reaching an invalid state, even mid-transaction
Isolation	Derby uses page-level locking to ensure uncommitted transaction changes are not visible to concurrent operations
Durability	Derby's Write-Ahead Log flushes committed transactions to disk before returning success, guaranteeing they survive a JVM crash

TransactionService.java — Code

```
package com.bookstore.transaction;

import com.bookstore.connection.ConnectionManager;
import java.sql.*;
import java.time.LocalDate;

public class TransactionService {

    private final ConnectionManager connManager;

    public TransactionService(ConnectionManager connManager) {
        this.connManager = connManager;
    }

    /**
     * Issues a product purchase to a customer.
     * Steps (all-or-nothing):
     * 1. Verify product availability
     * 2. Insert order record ← Savepoint AFTER this step
     * 3. Update Products.StockCount
     * 4. Update Customers.ActiveOrders ← Partial rollback to savepoint if
     fails
     */
    public int processPurchase(int productId, int customerId) throws
        SQLException {
        Connection conn = connManager.getConnection();
        conn.setAutoCommit(false);
```

```

        Savepoint afterOrderInsert = null;
        int orderId = -1;
        try {
            // Step 1: Verify availability
            int stock = getStockCount(conn, productId);
            if (stock <= 0) { conn.rollback(); return -1; }

            // Step 2: Insert order
            LocalDate today = LocalDate.now();
            LocalDate delivery = today.plusDays(7);
            String sql = "INSERT INTO
Orders (CustomerID, ProductID, OrderDate, DeliveryDate, Status) "
                + "VALUES (?, ?, ?, ?, 'PENDING')";
            try (PreparedStatement ps = conn.prepareStatement(sql, new
String[]{"ORDERID"})) {
                ps.setInt(1, customerId); ps.setInt(2, productId);
                ps.setDate(3, java.sql.Date.valueOf(today));
                ps.setDate(4, java.sql.Date.valueOf(delivery));
                ps.executeUpdate();
                try (ResultSet keys = ps.getGeneratedKeys()) {
                    if (keys.next()) orderId = keys.getInt(1);
                }
            }

            // SAVEPOINT
            afterOrderInsert = conn.setSavepoint("AFTER_ORDER_INSERT");

            // Step 3: Decrement stock
            String upd = "UPDATE Products SET StockCount = StockCount - 1, "
                + "InStock = CASE WHEN StockCount - 1 > 0 THEN 1 ELSE 0
END "
                + "WHERE ProductID = ?";
            try (PreparedStatement ps = conn.prepareStatement(upd)) {
                ps.setInt(1, productId);
                if (ps.executeUpdate() == 0) throw new SQLException("Product not
found: " + productId);
            }

            // Step 4: Increment active orders
            String upd2 = "UPDATE Customers SET ActiveOrders = ActiveOrders + 1
WHERE CustomerID = ?";
            try (PreparedStatement ps = conn.prepareStatement(upd2)) {
                ps.setInt(1, customerId);
                if (ps.executeUpdate() == 0) {
                    conn.rollback(afterOrderInsert);
                    conn.rollback(); return -1;
                }
            }

            conn.commit();
            return orderId;
        } catch (SQLException e) {
            try { conn.rollback(); } catch (SQLException ignored) {}
            throw e;
        } finally {
            conn.setAutoCommit(true);
        }
    }
}

```

Phase 3 — Core Application Workflow

Overview

Phase 3 brings the application to life through a menu-driven command-line interface and a complete set of CRUD operations. While Phase 1 set up the database and Phase 2 ensured transactional safety, Phase 3 connects everything together for actual user interaction.

Customer Management

The system allows new customers to be registered by capturing their full name, email, and phone number. Registered customers can be listed in a formatted table showing all details including their current active order count. Individual customers can also be looked up directly by their ID.

Product Catalog Management

New products can be added to the catalog by providing the title, author, ISBN, genre, price, and number of copies. The full product catalog can be displayed showing availability and stock counts. A filtered view showing only currently in-stock products is provided. Products can be searched directly by their ISBN using the index created in Phase 1.

Order Processing

Customers can purchase available products by providing a product ID and customer ID. Before processing, the system checks whether the customer already has a pending order for the same product to prevent duplicates. Members can cancel orders by providing their order ID.

BusinessLogic.java — Code (Key Methods)

```
package com.bookstore.business;

import com.bookstore.connection.ConnectionManager;
import com.bookstore.model.*;
import java.sql.*;
import java.time.LocalDate;
import java.util.*;

public class BusinessLogic {

    private final ConnectionManager connManager;

    public BusinessLogic(ConnectionManager connManager) {
        this.connManager = connManager;
    }

    /** Registers a new customer. Returns generated CustomerID. */
    public int registerCustomer(String name, String email, String phone) throws
    SQLException {
        String sql = "INSERT INTO
Customers(FullName,Email,Phone,RegisterDate,ActiveOrders) VALUES(?,?,?,?,0)";
        Connection conn = connManager.getConnection();
        try (PreparedStatement ps = conn.prepareStatement(sql, new
String[]{"CUSTOMERID"})) {
            ps.setString(1, name); ps.setString(2, email); ps.setString(3,
phone);
            ps.setDate(4, java.sql.Date.valueOf(LocalDate.now()));
```



```

        ps.executeUpdate();
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) return keys.getInt(1);
        }
    }
    return -1;
}

/** Returns all in-stock products ordered by Title. */
public List<Product> getAvailableProducts() throws SQLException {
    List<Product> list = new ArrayList<>();
    String sql = "SELECT * FROM Products WHERE StockCount > 0 ORDER BY
Title";
    try (PreparedStatement ps =
connManager.getConnection().prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
        while (rs.next()) list.add(mapProduct(rs));
    }
    return list;
}

/** Checks for a duplicate pending order. */
public boolean hasDuplicateOrder(int customerId, int productId) throws
SQLException {
    String sql = "SELECT COUNT(*) FROM Orders WHERE CustomerID=? AND
ProductID=? AND Status='PENDING'";
    try (PreparedStatement ps =
connManager.getConnection().prepareStatement(sql)) {
        ps.setInt(1, customerId); ps.setInt(2, productId);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() && rs.getInt(1) > 0;
        }
    }
}
}

```

Phase 4 — Performance Evaluation Framework

Overview

Phase 4 introduces a dedicated benchmarking module that empirically measures and compares the performance of four different JDBC access patterns. Rather than relying on theoretical assumptions, the system actually executes each pattern, times it precisely, and produces a structured report showing execution time and throughput.

Warm-Up Phase

Before any benchmark timing begins, a warm-up phase is executed for each test. This stabilizes the JVM's JIT compiler and Derby's buffer cache, ensuring that the first timed run is not penalized by cold-start overhead.

Benchmark 1 — Insert Strategy

Compares two approaches to inserting large volumes of records at both 1,000 and 10,000 record scales:

- Individual Insert: Calls `executeUpdate()` individually for each record, committing after every insert.

- Batch Insert: Uses `addBatch()` to queue multiple records and `executeBatch()` to send them all in one call with a single commit.

Benchmark 2 — Query Strategy

Compares a full-table scan against a B-tree indexed lookup on a 5,000-row benchmark table. The indexed lookup uses the index on CustomerID to navigate directly to matching rows.

Benchmark 3 — Statement Type

Compares string-concatenated Statement execution against compiled PreparedStatement execution across 500 insert operations. Every Statement call forces Derby to parse, plan, and compile the SQL from scratch, whereas PreparedStatement compiles once and reuses the plan.

Benchmark 4 — Transaction Granularity

Compares committing after every individual operation against issuing a single commit after all 100 operations complete. Every commit forces Derby to flush its Write-Ahead Log to disk — per-operation commits cause 100 disk flushes while batched commit causes only one.

PerformanceEvaluator.java — Key Code

```
package com.bookstore.benchmark;

import com.bookstore.connection.ConnectionManager;
import java.sql.*;
import java.util.*;

public class PerformanceEvaluator {

    private static final int RUNS = 5;
    private static final int WARMUP_ITERATIONS = 3;
    private final ConnectionManager connManager;
    private final List<BenchmarkResult> results = new ArrayList<>();

    public PerformanceEvaluator(ConnectionManager connManager) {
        this.connManager = connManager;
    }

    public void runAllBenchmarks() throws SQLException {
        results.clear();
        benchmark1_InsertStrategies();
        benchmark2_QueryStrategies();
        benchmark3_StatementTypes();
        benchmark4_TransactionGranularity();
        generateReport();
    }

    private void batchInsert(int count) throws SQLException {
        Connection conn = connManager.getConnection();
        conn.setAutoCommit(false);
        String sql = "INSERT INTO BenchmarkTable(DataValue, Category)
VALUES(?,?)";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            for (int i = 0; i < count; i++) {
```

```

        ps.setString(1, "Val_" + i); ps.setInt(2, i % 10);
        ps.addBatch();
        if (i % 500 == 0) ps.executeBatch();
    }
    ps.executeBatch();
    conn.commit();
} catch (SQLException e) { conn.rollback(); throw e; }
finally { conn.setAutoCommit(true); }
}
}

```

Phase 5 — Documentation, Cleanup & Validation

Overview

Phase 5 focuses on meaningful documentation, graceful resource cleanup, and thorough edge case validation to ensure the system is properly understood, properly closed, and properly tested against boundary conditions.

Edge Case Validation

Scenario	Handling
Duplicate Customer Email	Derby unique constraint on Email column; caught gracefully with user-friendly message
Duplicate Order	hasDuplicateOrder() check before transaction begins; rejected without touching any table
Out-of-Stock Product	Availability check at start of processPurchase(); rejected before any insert
Invalid Customer/Product ID	Foreign key constraint in Orders table rejects insert; transaction rolls back cleanly
Duplicate ISBN	Unique constraint on ISBN column; demonstrateConstraintViolation() explicitly tests this
Cancellation of Non-Pending Order	Zero-row update detected; processRefund() reports failure without committing

Resource Cleanup

Every PreparedStatement and ResultSet is opened inside a try-with-resources block guaranteeing automatic closure even on exceptions. The ConnectionManager's shutdown() method is called from MainApp's finally block, ensuring Derby shuts down cleanly whether the application exits normally or crashes.

Chapter 7: Snapshots

The following snapshots illustrate the system running live, covering all major menu flows including customer registration, product catalog management, order processing, ACID transaction demonstrations, and performance benchmarking.

7.1 Main Menu

```
----- MAIN MENU -----
1. Customer Management
2. Product Catalog
3. Order Processing (Purchase / Refund)
4. Query Reports
5. Transaction Demonstrations (ACID)
6. Run Performance Benchmarks
7. Verify Indexes (DatabaseMetaData)
8. Exit
-----
```

7.2 Customer Registration & Listing

```
Enter choice: 1
--- Customer Management ---
  1. Register new customer
  2. List all customers
  3. View customer by ID
Choice: 1
Name: Arjun Mehta
Email: arjun.mehta@gmail.com
Phone: 9876543210
[BusinessLogic] Customer registered: Arjun Mehta (ID=6)
✓ Customer registered with ID=6

--- Customer List ---
ID   Name           Email                Phone           Orders
1    Riya Sharma     riya@example.com     9876543210     0
2    Karan Patel     karan@example.com     9123456780     0
3    Priya Nair      priya@example.com     9988776655     0
4    Amit Verma      amit@example.com      8877665544     1
5    Sneha Das       sneha@example.com     7766554433     0
6    Arjun Mehta     arjun.mehta@gmail.com 9876543210     0
Total customers: 6
```

7.3 Product Catalog

```
--- Product Catalog ---
ID   Title           Author              ISBN              Genre
Avail/Total
1    Clean Code       Robert C. Martin    978-0-13-235088-4
Technology 3/ 3
2    The Pragmatic Programmer David Thomas        978-0-13-595705-9
Technology 2/ 2
3    Atomic Habits    James Clear         978-0-73-521129-2 Self-
Help      4/ 4
```

4	Sapiens	Yuval Noah Harari	978-0-06-231609-7
History	2/ 2		
5	Deep Work	Cal Newport	978-1-45-554412-0 Self-
Help	3/ 3		
6	1984	George Orwell	978-0-45-228423-4
Fiction	4/ 4		
7	Effective Java	Joshua Bloch	978-0-13-468599-1
Technology	2/ 2		
8	Thinking, Fast and Slow	Daniel Kahneman	978-0-37-453355-7
Psychology	2/ 2		
9	Introduction to Algorithms	Cormen et al.	978-0-26-204630-5
Technology	2/ 2		
10	To Kill a Mockingbird	Harper Lee	978-0-44-631078-1
Fiction	3/ 3		
Total: 10 product(s).			

7.4 Order Processing

```

— Order Processing —
  1. Purchase a product
  2. Cancel an order
  3. View all orders
Choice: 1
Product ID: 3
Customer ID: 6
[TransactionService] Savepoint 'AFTER_ORDER_INSERT' set (OrderID=5).
[TransactionService] Purchase committed. OrderID=5
✓ Purchase processed. Order ID=5
  Delivery date: 2026-05-26

Choice: 2
Order ID to cancel: 5
[TransactionService] Refund committed for OrderID=5
✓ Order cancelled successfully.

```

7.5 ACID Transaction Demonstrations

```

— ACID Transaction Demonstrations —
  1. Savepoint demo (partial rollback)
  2. Constraint violation + full rollback

Choice: 1



SAVEPOINT DEMONSTRATION



[Demo] Step A: Order inserted.
[Demo] Savepoint 'DEMO_SAVEPOINT' set.
[Demo] Step B: Updated 0 rows (expected 0 → triggers partial rollback).
[Demo] Rolled back to savepoint — Step B undone, Step A still in memory.
[Demo] Full rollback — Step A also undone. Database unchanged.
[Demo] ACID Integrity: PRESERVED ✓

Choice: 2



CONSTRAINT VIOLATION DEMO



[Demo] First insert succeeded.

```

```
[Demo] Caught expected constraint violation: duplicate ISBN.  
[Demo] Full rollback executed. No duplicate data in DB.  
[Demo] Database integrity: PRESERVED ✓
```

7.6 Performance Benchmarks

```
PERFORMANCE REPORT -- JDBC BENCHMARK SUITE (Apache Derby)  
=====
```

Operation	Records	Avg(ms)	StdDev	Tput/s
Individual INSERT	1000	37.05	12.94	26,991
Batch INSERT	1000	22.72	2.94	44,014
Individual INSERT	10000	148.84	20.65	67,187
Batch INSERT	10000	108.46	18.86	92,203
Full-Table Scan (Orders)	5000	4.69	0.07	1,066,976
Indexed Lookup (CustomerID)	5000	1.16	0.27	4,311,906
Plain Statement (string concat)	500	437.93	177.13	1,142
PreparedStatement (compiled)	500	7.44	3.27	67,172
Per-operation Commit	100	12.20	2.94	8,197
Batched Commit (all 100 ops)	100	2.03	0.53	49,184

```
=====
```

OBSERVATIONS:

- * Batch INSERT is ~1.75x faster (fewer round-trips + single commit overhead)
- * Indexed lookup is ~4x faster than full-table scan at 5,000 rows
- * PreparedStatement outperforms Statement by ~59x (query compiled once, reused)
- * Batched commit is ~6x faster (one WAL flush vs one per operation)

```
=====
```

[PerformanceEvaluator] Report saved to performance_report.csv

Chapter 8: Conclusion

This project successfully designed, implemented, and benchmarked a complete end-to-end JDBC application in the form of an Online Bookstore Management System built on Java and Apache Derby in embedded mode. Across five structured phases, the project demonstrated that professional-grade database application development requires equal attention to correctness, performance, and code quality — and that these goals reinforce rather than conflict with each other.

Database Foundation

Phase 1 established that a well-designed schema is the bedrock of a reliable application. By defining constraints, foreign keys, and indexes at the database level rather than relying solely on application-level validation, the system ensures data integrity independent of how the application behaves. Derby's embedded mode proved to be a practical and self-contained choice that eliminated all external infrastructure dependencies while providing full relational database capabilities.

Transaction Management

Phase 2 demonstrated that explicit transaction control is not optional in any application that modifies multiple related tables. The `processPurchase()` and `processRefund()` operations showed that without atomic commit and rollback boundaries, even a simple purchase operation can corrupt three tables simultaneously if interrupted mid-execution. Named savepoints added a layer of recovery sophistication, allowing partial rollback without discarding entire transactions. All four ACID properties were demonstrated through working code and deliberate failure scenarios.

Application Functionality

Phase 3 confirmed that clean architectural separation between the data access layer, transaction layer, and presentation layer produces code that is easier to maintain, test, and extend. `BusinessLogic` handled all CRUD operations through parameterized `PreparedStatement`s, `MainApp` handled all user interaction, and `TransactionService` handled all multi-step atomic operations — each layer focused on a single responsibility without overlapping concerns.

Performance Findings

Phase 4 produced the most quantitatively striking results of the project. `PreparedStatement` outperformed string-concatenated `Statement` by approximately 59x, batched commits outperformed per-operation commits by approximately 6x, and indexed lookups outperformed full-table scans by approximately 4x. Crucially, the safest choices — `PreparedStatement` over `Statement`, explicit transactions over auto-commit — were simultaneously the fastest choices, proving that security and performance point in the same direction in a well-designed JDBC application.

Validation and Documentation

Phase 5 confirmed that a production-quality application must account for failure at every step. Duplicate registrations, out-of-stock products, invalid references, constraint violations mid-transaction, and repeated cancellation attempts were all tested and handled gracefully. The analysis document and README together provide a complete record of the system's design decisions, behavioral guarantees, and performance characteristics for any future developer or evaluator.

Chapter 9: References

1. Official Documentation

- Oracle Corporation. Java SE 11 Documentation — java.sql Package (JDBC API). <https://docs.oracle.com/en/java/docs/>
- Apache Software Foundation. Apache Derby 10.14 Developer's Guide. <https://db.apache.org/derby/docs/10.14/devguide/>
- Apache Software Foundation. Apache Derby 10.14 Reference Manual. <https://db.apache.org/derby/docs/10.14/ref/>

2. Books

- Bloch, J. Effective Java. 3rd ed. Addison-Wesley Professional, 2018.
- Martin, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.
- Thomas, D. and Hunt, A. The Pragmatic Programmer: Your Journey to Mastery. 20th Anniversary ed. Addison-Wesley Professional, 2019.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional, 1994.

3. JDBC and Database Concepts

- Oracle Corporation. JDBC Database Access — The Java Tutorials. <https://docs.oracle.com/javase/tutorial/jdbc/>
- Oracle Corporation. Using Transactions in JDBC. <https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html>
- Oracle Corporation. Using Prepared Statements. <https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html>
- Oracle Corporation. Using Batch Updates. <https://docs.oracle.com/javase/tutorial/jdbc/basics/batchupdates.html>

4. Transaction Management and ACID Properties

- Ramakrishnan, R. and Gehrke, J. Database Management Systems. 3rd ed. McGraw-Hill, 2002.
- Silberschatz, A., Korth, H. F. and Sudarshan, S. Database System Concepts. 7th ed. McGraw-Hill, 2019.
- Gray, J. and Reuter, A. Transaction Processing: Concepts and Techniques. Morgan Kaufmann, 1992.

5. Performance and Indexing

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. Introduction to Algorithms. 3rd ed. MIT Press, 2009.
- Apache Software Foundation. Apache Derby Performance Tuning Guide. <https://db.apache.org/derby/docs/10.14/tuning/>

6. Tools and Environment

- Oracle Corporation. Java Development Kit (JDK) 11.
<https://www.oracle.com/java/technologies/javase/jdk11-archive-downloads.html>
- Apache Software Foundation. Apache Derby Downloads.
https://db.apache.org/derby/derby_downloads.html

GitHub Repository:

https://github.com/subhamsahoo77037droid/Subham_Kumar_Sahoo_2341019060_35